APIGUARD: A Machine Learning-Assisted Framework for Continuous API Security Assessment in DevSecOps Pipelines

Deliverables 2 and 3 — System Model and Proposed Methodology

Thornkasidej Fuangfoo (6622771234) Varit Mesri (6622771317)
Sirindhorn International Institute of Technology (SIIT), Thammasat University, Pathum Thani, Thailand
Adviser: Somchart Fugkeaw

***Abstract*—Rule-based linting is the only security check most OpenAPI specifications receive before deployment. A linter fires when a written rule matches, which leaves it unable to say which findings matter, unable to distinguish an endpoint returning a health-check string from one returning customer records, and silent about anything no rule covers. In a CI/CD pipeline this produces alert fatigue and blind spots at the same time. This document specifies the system model of APIGUARD, a framework that places a supervised classifier *after* the rule scanner rather than in place of it. The scanner keeps its authority over what rules cover; the model assigns every endpoint a risk score, ranks the findings, and explains each score with SHAP attributions, so that the pipeline gate acts on a risk score rather than on an alert count. We define the entities and their trust relationships, the notation and the gate decision rule, every runtime and offline component with its inputs, outputs and guarantees, the data flow of a single pipeline run, the threat model and its stated limits, and the deployment form. We also map each research question to the experiment that will answer it. The methodology section then specifies how labels are made and kept apart from the scanner's own output, the training algorithm, a gate that judges only new or changed endpoints, and the rule hold-out protocol that tests whether the model generalises beyond the rules.**

***Index Terms*—API security, OpenAPI, DevSecOps, CI/CD, static analysis, XGBoost, SHAP, OWASP API Security Top 10, risk scoring.**

I. Introduction

Modern software is assembled out of HTTP APIs, and the OpenAPI specification is the first place where an API's security posture is written down: which endpoints require authentication, which scheme they use, what data comes back, and what limits apply. That makes the specification a cheap and early place to catch security design mistakes, long before a penetration test is run.

Today this checking is done by rules only. A linter such as Spectral with the OWASP ruleset fires when a written rule matches. It cannot say which of its findings actually matter, it produces the same alert whether an endpoint returns a health-check string or a customer's date of birth, and it says nothing about a problem nobody wrote a rule for. Inside a CI/CD pipeline this causes two failures at once: developers stop reading the alerts, and the gate lets dangerous designs through.

Scale makes the opportunity concrete. The APIstic dataset contains 1,275,568 valid OpenAPI specifications mined from GitHub, SwaggerHub, BigQuery and APIs.guru [1], and the curated APIs.guru directory alone holds 2,529 API descriptions covering 108,837 endpoints [8]. This is a large, machine-readable body of real-world API design, and it is almost untouched as training data for security.

APIGUARD places a supervised model *after* the rule scanner. The scanner keeps finding what rules find; the model gives every endpoint a risk score, ranks the findings, and explains each score with SHAP. The pipeline gate then acts on a risk score instead of a raw alert count. This document specifies the system model: the entities involved, the notation used, every component with its inputs and outputs, the data flow of a single pipeline run, the trust assumptions, and the way the system is deployed. It then specifies the proposed methodology and algorithms (Section XV).

The contributions are:

- A system model in which a classifier scores *every* endpoint after the rule scanner, so that the gate acts on a ranked, explained risk score instead of an alert count.
- A label protocol that keeps scanner labels for training only and measures every result against independently produced labels (L2 and L3), which removes the circularity of training and testing on scanner output.
- A rule hold-out protocol (E2) that tests whether the model learned a design property rather than a rule trigger.
- A gate that judges only new or changed endpoints, so that existing APIs can adopt it.
- A working implementation of the normaliser (S0), the feature builder (S2a, 66 features) and the gate (S3), each

This deliverable describes the system for the registered topic "Continuous API Security Assessment in DevSecOps Pipelines." It replaces the Infrastructure-as-Code system presented in Deliverable 1, which was withdrawn because that topic had already been reserved by another group. The target artefact is now the OpenAPI specification instead of Terraform code.

with contract tests.

Changes since Deliverable 2. The system model of Deliverable 2 is kept and revised as follows: (i) a fabricated author name was removed from the VAmPI reference; (ii) the SHAP contract now states that attributions sum to the raw margin, not to the score; (iii) the claim that a developer cannot change the threshold now names the mechanism that enforces it; (iv) the evaluation no longer scores the model against the scanner labels it trained on; (v) a language-model baseline and a text-inflation test were added; (vi) the feature table now matches the implemented feature builder; and (vii) the gate counts only new or changed endpoints.

II. Related Work

A. Analysis of OpenAPI Specifications

Serbout and Pautasso collected 1,275,568 OpenAPI specifications and published structural metrics for them [1]; their work describes how APIs are written, not how safe the designs are. Decrop *et al.* automate the quality analysis of OpenAPI specifications [4]. The closest work to ours, Cheh and Chen, analyse OpenAPI documents for security design issues with a graph model of sensitive-field exposure, but evaluate it on a single API [2]. Tanveer *et al.* survey the detection of vulnerabilities in RESTful APIs [3].

B. Rule-Based Scanning and Alert Prioritisation

Spectral with the OWASP ruleset turns the API Security Top 10 into rules that fire on a specification [9], [13]. A rule fires or it does not; it gives no ranking and cannot report a problem nobody wrote a rule for. Flynn *et al.* show that classification models can prioritise the alerts of several static analysis tools [5]. APIGUARD applies this idea at endpoint level to API specifications, and additionally scores endpoints on which no rule fired.

C. Weak Labels and Explanation

Training on automatically produced labels is the weak-supervision setting of Snorkel [18]. Because such labels are noisy, we use them for training only. Gradient-boosted trees [6] are explained with SHAP [7] and its exact tree algorithm [14]. Leakage between training and test data is a documented cause of over-optimistic results [19], which is why every split here is grouped by provider.

D. Gap

To our knowledge, no existing work combines a learned, explained, endpoint-level risk score with a pipeline gate and tests it on rules that were held out of training. This is the gap APIGUARD addresses.

III. System Overview

APIGUARD is organised into three lanes. Separating them is a design decision rather than a drawing convenience: each lane runs at a different time, under different trust conditions, and at a different speed.

- The **runtime lane** is what ships. It runs on every commit inside the CI/CD pipeline, must finish in seconds, and is fully automatic.
- The **offline lane** builds the model. It runs rarely, on a workstation or build server, and may take hours.
- The **feedback lane** is how a human verdict re-enters the system. It is deliberately slow and never reaches the model directly.

Fig. 1 shows the three lanes and the two connections between them: the trained model artefact loaded by the runtime, and the shared code path that guarantees training and runtime compute features identically.

IV. Entities, Roles and Trust Assumptions

Six entities take part in the system. Table I states the role of each and how far it is trusted.

V. Notation and Definitions

Table II fixes the notation used here and in every later deliverable.

For one specification S, the runtime lane computes the scored endpoint set

$$R(S) = \{\, (e,\ r(e),\ \phi(e)) \ :\ e \in E(S) \,\}, \tag{1}$$

and the gate decision is

$$G(S) = \begin{cases} 1, & \left|\{\, e \in E(S) \ :\ r(e) \geq \tau \,\}\right| > k, \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

With the default policy $k = 0$ the gate fails if any single endpoint scores at or above τ. A larger k lets a team adopt the tool gradually on an existing API without blocking every build on the first day. The value of τ is not chosen by hand: it is swept on a validation set and reported as a precision/recall trade-off curve (Experiment E3).

Two properties the design must hold. First, $r(e)$ is defined for *every* endpoint, including endpoints on which no rule fired; for those, the scanner-derived features are simply zero. Second, for a fixed specification, ruleset version and model artefact the whole runtime lane is deterministic — nothing in it samples randomly.

VI. Runtime Lane: Components S0–S3

A. S0 — Normaliser

Input: a specification file in YAML or JSON, either OpenAPI 3.x or Swagger 2.0. **Output:** a list of normalised endpoint records, one per (path, method) pair.

The normaliser resolves internal `$ref` pointers, expands the schema of every request body and response, and flattens the two specification versions into one internal form, so that no component after S0 needs to know which version the document came from. Path-level parameters are merged into each operation, and a global `security` block is propagated to operations that do not declare their own — recording, importantly, whether the operation *inherited* or *overrode* that requirement. An operation that explicitly overrides global

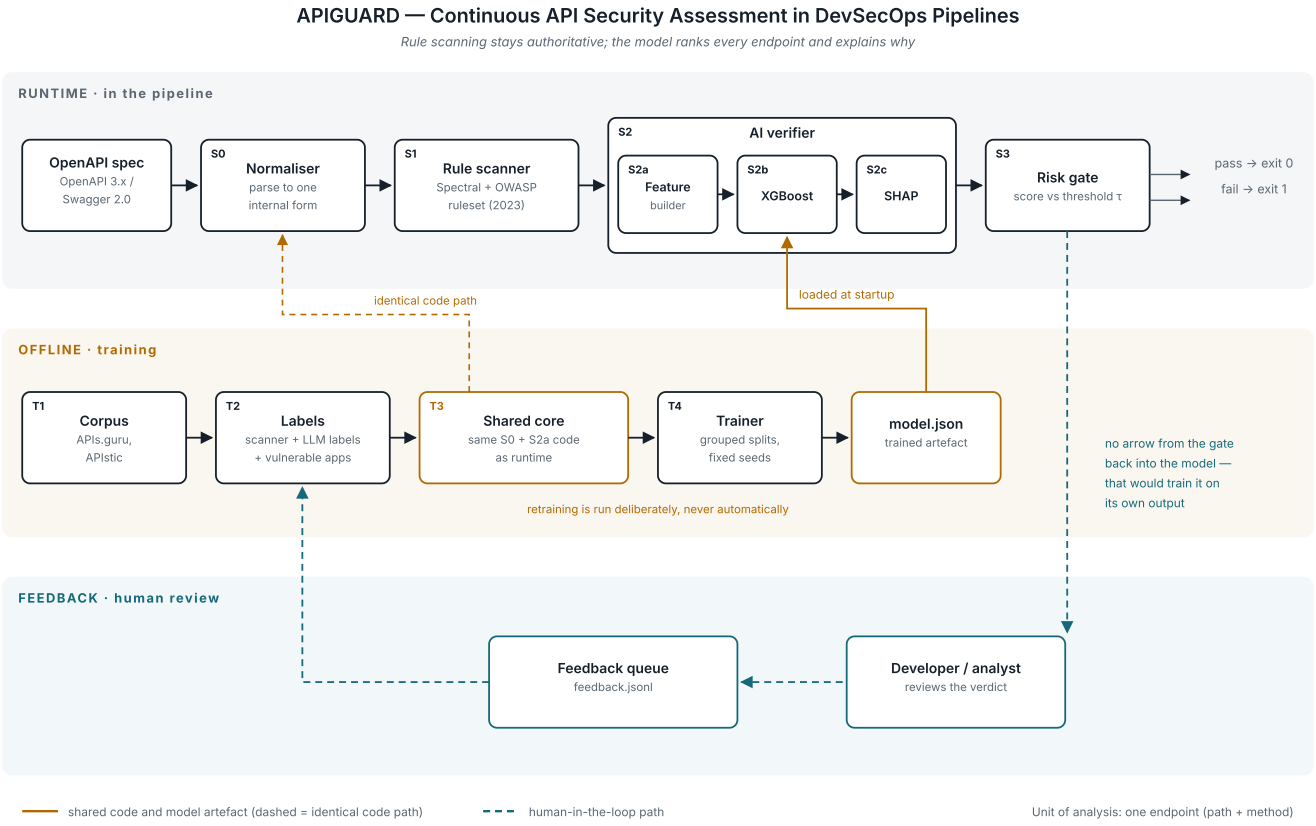


Fig. 1. APIGUARD system model. The runtime lane (S0–S3) runs inside the pipeline; the offline lane (T1–T4) produces `model.json`; the feedback lane returns human verdicts to the label store. Amber marks shared code and the model artefact; dashed teal marks the human-in-the-loop path. The unit of analysis is one endpoint.

TABLE I
ENTITIES, THEIR ROLES AND TRUST ASSUMPTIONS

Entity	Role in the system	Trust assumption
Developer	Writes or edits an OpenAPI specification and pushes it to the repository; reads the gate output and fixes what it reports.	Not malicious, but assumed to make design mistakes. Cannot change the gate threshold on a protected branch: the workflow file is guarded by branch protection or an organisation-level reusable workflow.
CI/CD runner	Checks out the repository, invokes the APIGUARD CLI, and acts on its exit code.	Trusted execution environment; its configuration is under version control.
APIGUARD CLI	The runtime lane, S0–S3. Reads a specification and produces a per-endpoint risk report and a pass/fail exit code.	The component under study. Must be deterministic for a fixed specification, ruleset version and model.
Rule scanner	Third-party linter (Spectral with the OWASP ruleset). Supplies findings used as a baseline and as model input.	Trusted but incomplete. Version is pinned, because a version change silently changes every label.
Security analyst (here, the two team members)	Reviews a sample of gate verdicts, marks each correct or incorrect, and owns the retraining decision.	Authoritative for labels. The only entity allowed to change what the model learns.
Offline trainer	Builds the corpus, assembles labels, trains the model and emits `model.json`.	Runs outside the pipeline. Never invoked automatically by a gate decision.

security with an empty requirement is a different risk from one that simply inherits it, and a design that collapsed the two would lose that distinction before the model ever saw it.

Guarantees. Every endpoint of a parsable specification appears exactly once in the output. A specification that cannot be parsed is not silently skipped: it is written to an error log with the reason, and the corpus statistics report the parse-failure rate. Circular references are cut at a fixed depth so that expansion always terminates.

B. S1 — Rule Scanner

Input: the original specification file. **Output:** the set $A(e)$ of findings for each endpoint.

TABLE II
NOTATION USED THROUGHOUT THE PROJECT

Symbol	Meaning
S	An OpenAPI 3.x or Swagger 2.0 specification document; the input to one pipeline run.
$e = (p, m)$	One *endpoint*: a path p combined with an HTTP method m. This is the unit of analysis.
$E(S)$	The set of all endpoints declared in S, produced by the normaliser S0.
$A(e)$	The scanner findings attributed to e, each carrying a rule identifier, a severity and an OWASP API Top 10 category.
$x(e) \in \mathbb{R}^d$	The feature vector of e, produced by S2a from the normalised specification and from $A(e)$.
$y(e) \in \{0, 1\}$	The label of e: 1 if the endpoint carries a security-relevant design issue, 0 otherwise.
$f : \mathbb{R}^d \to [0, 1]$	The trained classifier (XGBoost), loaded from `model.json` at CLI startup.
$r(e)$	The risk score of e, defined as $r(e) = f(x(e))$.
$\phi(e)$	The SHAP attribution vector for e: the contribution of each feature to the raw margin (log-odds) from which $r(e)$ is computed.
$\tau,\ k$	The gate threshold and the tolerated count of endpoints at or above it.
$G(S)$	The gate decision for the whole specification: 0 = pass, 1 = fail.

S1 runs Spectral with `@stoplight/spectral-owasp-ruleset` v2.x, which implements the OWASP API Security Top 10, 2023 edition [9], [13]. Spectral reports findings against JSON paths inside the document; S1 maps each finding down to the endpoint it belongs to, so that a document-level finding is attributed to every endpoint and an operation-level finding only to its own. The 42Crunch audit score may optionally be collected as an additional signal.

S1 output is used in three distinct ways, and keeping them apart matters for the evaluation: it is the **baseline** the model is compared against, it is the source of the **automatic labels** used for training, and it enters the model as **input features**. The scanner and ruleset versions are pinned and recorded in every report, because a ruleset upgrade changes both the baseline and the labels at the same time.

C. S2 — AI Verifier

S2 is the core of the contribution and has three subcomponents.

S2a Feature builder. Turns one normalised endpoint record, plus its scanner findings, into a fixed-length numeric vector $x(e)$. The features fall into the six groups of Table III. The builder is a tested module with no dependency on the training pipeline, which is what allows the same code to be imported by both S2a and T3.

S2b Classifier. An XGBoost gradient-boosted tree ensemble [6], loaded from `model.json` at startup and applied to every endpoint. Gradient boosting is chosen because the input is a table of mixed numeric, binary and categorical features, because strong class imbalance is handled directly through `scale_pos_weight`, and because tree ensembles remain traceable back to individual features. Simpler baselines — majority class, logistic regression and random forest — are trained on the identical feature table for comparison.

S2c Explainer. SHAP [7] produces $\phi(e)$, attributing the score to individual features using Shapley values from cooperative game theory, computed exactly for tree ensembles by TreeSHAP [14]. Every finding APIGUARD reports carries the three to five features that pushed its score up. This is not decoration: it is how a developer checks whether the model reasoned about a security property or about something irrelevant, and it is how the team detects a model that has learned a shortcut.

D. S3 — Risk Gate

Input: $R(S)$, the scored endpoint set. **Output:** a report and a process exit code.

S3 applies the decision rule of (2) and emits the ranked findings in one of three formats: human-readable text for a terminal, JSON for further processing, and SARIF so that results appear as annotations in the code-review interface. The exit code is 0 on pass and 1 on fail, which is the only part of the system the CI runner needs to understand. The report always lists endpoints in descending risk order, and each entry carries the score, the SHAP reasons, and any scanner findings that fired.

VII. OFFLINE LANE: COMPONENTS T1–T4

A. T1 — Corpus

Table IV lists the four sources. Each downloaded specification is stored with its source URL and a snapshot date, and is de-duplicated by normalised content hash before anything else happens. Public API specifications are heavily copied and templated between projects; near-duplicates left in the corpus would appear on both sides of a train/test split.

B. T2 — Label Store

Three label sources are kept separate and never merged into one undifferentiated pile.

- **L1 automatic.** Spectral OWASP findings mapped down to endpoints. Cheap and plentiful, and circular if used alone — a model trained only on L1 can at best reproduce the scanner.
- **L2 reference labels.** A stratified sample of 304 endpoints labelled independently by three language-model passes that never saw scanner output or model scores, combined by majority vote, with agreement reported as Cohen's and Fleiss' κ. No human labelled these endpoints. Small and imperfect, but independent of the scanner, which is what the false-positive claim needs.
- **L3 ground truth.** VAmPI and crAPI endpoints with known, documented vulnerabilities.

TABLE III
FEATURE GROUPS COMPUTED BY S2A FROM ONE NORMALISED ENDPOINT

Group	Contents	Why it should carry signal
A. Structural	Path depth; number of path parameters; whether a path parameter looks like an object identifier (`id`, `uuid`, `_id`); HTTP method; query and header parameter counts; request body present; response schema property count; declared status codes; whether 401/403 are declared; deprecated flag.	An object identifier in the path with no ownership check is the classic API1 Broken Object Level Authorization shape.
B. Security scheme	Whether the endpoint has a security requirement; inherited versus overridden; scheme type (apiKey, http-basic, http-bearer, oauth2, openIdConnect); scope count; whether the server URL uses `http` rather than `https`.	The direct expression of authentication and authorisation design (API2, API5).
C. Data sensitivity	Count of response property names matching a sensitive-term lexicon (password, token, secret, ssn, dob, phone, email, address, card, balance); maximum term weight; sensitive properties in response versus request; ratio of sensitive to total properties; the combination flag *no security requirement and sensitive response fields*.	What a rule cannot see: it separates a health-check endpoint from one returning customer data (API3).
D. Documentation hygiene	Description and summary lengths; example present; share of parameters with a description or example; version segment in the path; pagination parameters present; rate-limit headers documented.	Poorly specified endpoints correlate with weak design; rate-limit documentation relates to API4.
E. Scanner-derived	`flagged_by_spectral`, `rule_count`, `severity_max`, `owasp_categories_hit`, optionally `42crunch_audit_score`. All zero when nothing fired.	Lets the model use the rules without depending on them. Kept separable so the ablation can remove the group entirely.
F. Context	Endpoint count of the parent specification; whether the file is OpenAPI 3.x or Swagger 2.0. Provider identity is deliberately excluded and used only to group the splits.	Places the endpoint inside its API rather than judging it in isolation.

TABLE IV
CORPORA AND THEIR ROLE IN THE PROJECT

Source	Size	Use
APIs.guru [8]	2,529 APIs, 108,837 endpoints	Primary corpus; curated, clean, version-stable.
APIstic [1]	1,275,568 specifications	Scale-up if the schedule allows; reference for structural metrics.
VAmPI [10]	1 API, vulnerable *and* secure mode	Ground truth; the same API in two configurations gives matched positive and negative examples.
OWASP crAPI [11]	1 API, documented challenges	Second ground-truth target, independent of VAmPI.

C. T3 — Shared Core

T3 is not a separate implementation. It is the S0 normaliser and the S2a feature builder *imported* by the training pipeline, not copied into it. This is a deliberate constraint: if training and runtime computed features from two code paths, any drift between them would appear as an unexplained drop in live accuracy that no experiment could diagnose. The amber path in Fig. 1 marks this shared code.

D. T4 — Trainer

T4 takes the labelled feature table and produces `model.json`. Four rules govern it.

1) **Split by provider or repository, never at random.** A random split leaks near-identical endpoints across the boundary and produces impressive but meaningless scores. Grouped 5-fold cross-validation is used throughout.
2) **Report PR-AUC, F1 and MCC, not accuracy.** Insecure endpoints are a small minority; a model that predicted "secure" for everything would score high on accuracy alone.
3) **Fix every random seed and pin every dependency version.** The scanner version, ruleset version, corpus snapshot date and seed are written into the model artefact's metadata.
4) **Emit an artefact, not a live service.** `model.json` is a file under version control, so any reported result can be reproduced with the exact model that produced it.

VIII. FEEDBACK LANE

When a developer or analyst disagrees with a verdict, the decision is appended to `feedback.jsonl` as one record per endpoint: the endpoint identity, the specification hash, the score it received, the human verdict, the reviewer and a timestamp. The queue is an input to T2 at the *next deliberate retrain*, run by a person who has looked at what is in the queue.

There is deliberately **no arrow from S3 back into S2b**. A model that consumes its own accepted predictions as training data reinforces its own mistakes and drifts away from the ground truth, with the drift hidden by the model's own agreement with itself. Human verdicts are therefore treated as new labels, not as confirmation.

IX. END-TO-END DATA FLOW

One pipeline run, in order:

1) A developer pushes a commit that touches `openapi.yaml`.
2) The CI runner checks out the repository and invokes `apiguard scan openapi.yaml --threshold 0.70`.
3) **S0** parses the file, resolves references and emits $E(S)$ — say 48 endpoint records.
4) **S1** runs Spectral with the pinned OWASP ruleset and attributes each finding to an endpoint, giving $A(e)$. Suppose 11 endpoints are flagged.

5) **S2a** builds $x(e)$ for all 48 endpoints, not only the 11. For the other 37, group E features are zero.
6) **S2b** loads `model.json` once and scores all 48, producing $r(e)$.
7) **S2c** computes $\phi(e)$ and keeps the top contributing features per endpoint.
8) **S3** ranks by $r(e)$, applies τ and k, writes the report and exits 0 or 1. An unflagged endpoint may appear at the top of the ranking — for example `GET /users/{id}` with no security requirement and a response containing `email` and `phone`.
9) The runner fails the build on exit code 1 and surfaces the SARIF annotations in the pull request.
10) The developer either fixes the specification or marks the finding as a false positive, which appends a record to `feedback.jsonl`.
11) At the next retrain, an analyst reviews the queue, T2 updates the labels, T4 produces a new `model.json`, and the new artefact is committed.

X. Interfaces and Data Contracts

Table V states what each component accepts, what it returns, and the contract it must satisfy. These contracts are what allow the components to be unit-tested in isolation.

XI. Assumptions and Threat Model

What is trusted. The CI runner and the repository are trusted. The specification file is trusted as an *input*, not as a *claim*: APIGUARD reads what it says but does not assume it is true of the running service.

Who the adversary is. The adversary is an external attacker interacting with the deployed API through its public interface, looking for endpoints that expose data without adequate authentication or authorisation, that leak sensitive fields, or that lack rate limiting. APIGUARD does not model an insider who tampers with the pipeline configuration, and it is not a defence against a developer who deliberately weakens the specification and lowers τ at the same time. The model can also be gamed: group D features can be inflated by writing longer prose, so Experiment E5 includes a text-inflation test (Section XV).

Assumptions. (i) The specification is a reasonably faithful description of the implemented API; where it is not, APIGUARD's finding is still useful as a documentation defect. (ii) Endpoints can be judged largely independently of one another; cross-endpoint attack chains are out of scope. (iii) The corpus of public specifications is representative enough of real-world API design to train on — an assumption tested directly by Experiment E4.

The central limitation, stated honestly. A specification can be secure on paper while the implementation behind it is not. APIGUARD measures *design-level* risk. It is not a substitute for dynamic testing and is not claimed to be. Experiment E4 measures how far design-level scoring carries into two deliberately vulnerable applications, and that measurement — whatever it shows — is reported as part of the result.

Out of scope. Source-code analysis of API implementations; GraphQL, gRPC and SOAP; live traffic analysis and WAF-style blocking; and dynamic scanning at corpus scale. Schemathesis is used only in Experiment E9, against one application, and never across the corpus.

XII. Design Justifications

Three decisions visible in Fig. 1 will be challenged in review, and each has a reason.

1) The model sits after the scanner, not instead of it. Rules stay authoritative for what they cover, and the model adds ranking plus coverage of what rules miss. This is the same arrangement Flynn *et al.* used to prioritise alerts from multiple static analysis tools with classification models [5]. The alternative — replacing the linter with a model — would trade a deterministic, auditable check for a probabilistic one, which is the wrong trade for a build gate.

2) S2 scores every endpoint, not only the flagged ones. Scanner output enters as features that are zero when nothing fired, so an endpoint no rule touched can still score high. Scoring only flagged endpoints would cap the model's recall at the scanner's recall and make the generalisation research question unanswerable.

3) Feedback never reaches the model directly. Human verdicts enter through the label store at the next deliberate retrain, for the reason given above.

4) A tree ensemble rather than a language model. A committed `model.json` gives the same score for the same input on every run, while a hosted language model can change between versions. Scoring 108,837 endpoints with a language model costs one call each, while XGBoost scores a specification in one batched prediction. A SHAP attribution names the features that moved a score, while a language model gives an explanation that sounds right but is not tied to the score. Nevertheless a zero-shot language model is evaluated as baseline B4 in Section XV, so the choice is tested and not assumed.

Position against related work. The related course topic "Automatic API Security Design Review for Microservices Using Threat Graph Analysis" performs threat modelling once, at design time, over a service graph. APIGUARD runs continuously inside the pipeline on every commit, at endpoint granularity, and produces a numeric gate decision with a per-finding explanation. The nearest published work, Cheh and Chen [2], builds a graph model of sensitive-field exposure in OpenAPI documents but evaluates it on a single API; recent surveys of RESTful API vulnerability detection [3] and of OpenAPI quality analysis [4] confirm that learned, explained risk scoring at corpus scale has not been attempted. That is the gap this project addresses.

XIII. Deployment View

APIGUARD ships as a command-line tool that reads a file and returns an exit code, so it fits any CI system without a dedicated integration. A GitHub Actions job is the reference deployment:

TABLE V
COMPONENT INTERFACES AND THEIR CONTRACTS

Component	Input	Output	Contract
S0	Specification file (YAML/JSON)	Normalised endpoint records	Every endpoint appears once; parse failures are logged, never dropped silently.
S1	Specification file	Findings attributed per endpoint	The ruleset version is recorded in the output.
S2a	Endpoint record and its findings	Fixed-length feature vector	Same length and ordering for every endpoint; no null values.
S2b	Feature vector	Score in $[0, 1]$	Deterministic for a fixed model artefact.
S2c	Feature vector and model	SHAP attributions	Attributions sum to the raw margin (log-odds) minus the base margin; $r(e)$ is the logistic transform of that margin.
S3	Scored endpoint set	Report and exit code	Exit code is 0 or 1 only; the report is always ranked by score.
T4	Labelled feature table	`model.json`	Carries seed, tool versions and corpus snapshot date in its metadata.

TABLE VI
RESEARCH QUESTIONS MAPPED TO EXPERIMENTS

Research question	Experiment	What is measured
RQ1 Feasibility. Does the model predict endpoint-level findings more accurately than the scanner alone?	E1 — provider-grouped 5-fold cross-validation, scored against reference labels (L2, L3)	PR-AUC, F1 and MCC against majority-class, scanner, logistic-regression, random-forest and language-model baselines.
RQ2 Generalisation. With one OWASP category removed from training, does the model still detect it?	E2 — rule hold-out (the headline experiment)	Recall on the held-out category's endpoints; above-chance recall means the model learned the design property, not the rule trigger.
RQ3 False positives. Does ranking reduce reading effort before the real issues?	E3 — analyst effort on the L2 reference-labelled sample	Precision@k and findings read to reach all true positives, versus scanner severity ordering; τ swept.
Supporting — external validity	E4 — VAmPI and crAPI	Whether known vulnerable endpoints rank above benign ones, and VAmPI secure mode below vulnerable mode.
Supporting — runtime	E3 (same run)	Wall-clock time of S0–S3 on specifications of 10 to 500 endpoints.
Supporting — which features carry the signal	E5 — ablation and SHAP	Performance with each feature group removed, especially group E; global SHAP importance.

```
- name: API security gate
  run: |
    apiguard scan api/openapi.yaml \
      --model models/model_l2plus.json \
      --threshold 0.70 \
      --max-above 0 \
      --format sarif --out apiguard.sarif
- uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: apiguard.sarif
```

The model artefact is committed to the repository alongside the pipeline definition, so a build is reproducible from the checkout alone. Target runtime is a few seconds for a specification of a few hundred endpoints: S1 dominates, since S2b is a single batched prediction over a feature table. The gate can be run in report-only mode by setting a large k, which is the recommended way to introduce it to an existing API before it is allowed to block a build.

XIV. HOW THE SYSTEM MODEL WILL BE VALIDATED

The system model above defines what the experiments must measure. Table VI maps each research question to the experiment that answers it.

E2 is the experiment that answers the obvious criticism of the whole design — that a model trained on scanner labels has only learned the scanner. It is protected in the schedule ahead of every optional component; the cut order is given in Section XV.

XV. PROPOSED METHODOLOGY AND ALGORITHM

This section turns the system model into a procedure that can be run and checked. It fixes what a label means, how labels are produced and kept honest, how features are computed, how the model is trained and tuned, how the gate judges a change, and how each experiment is carried out. Fig. 2 shows the order of the steps. One rule runs through all of them: *no result is ever measured against the scanner labels the model was trained on.*

A. Problem Statement and Label Definition

Given an endpoint e and its feature vector $x(e) \in \mathbb{R}^d$, learn a function f whose output $r(e) = f(x(e))$ ranks endpoints by the chance that they carry a security-relevant *design* issue. The label $y(e) = 1$ when the specification gives evidence of at least one issue in an OWASP API Top 10 category that a specification can show [13]: API1 (object-level authorisation), API2 (authentication), API3 (property-level exposure), API4 (resource consumption), API5 (function-level authorisation), API8 (misconfiguration) and API9 (improper inventory). API6, API7 and API10 rarely leave evidence in a specification; they are recorded as "not assessable" and are never counted as negatives.

B. Corpus Preparation

1) **Snapshot.** Download the APIs.guru directory once and store the snapshot date, the source URL of every file and the directory commit. All counts in the final report are counts of this snapshot.
2) **Parse.** Run S0 on every file. Files that fail are logged with a reason and the parse-failure rate is reported.

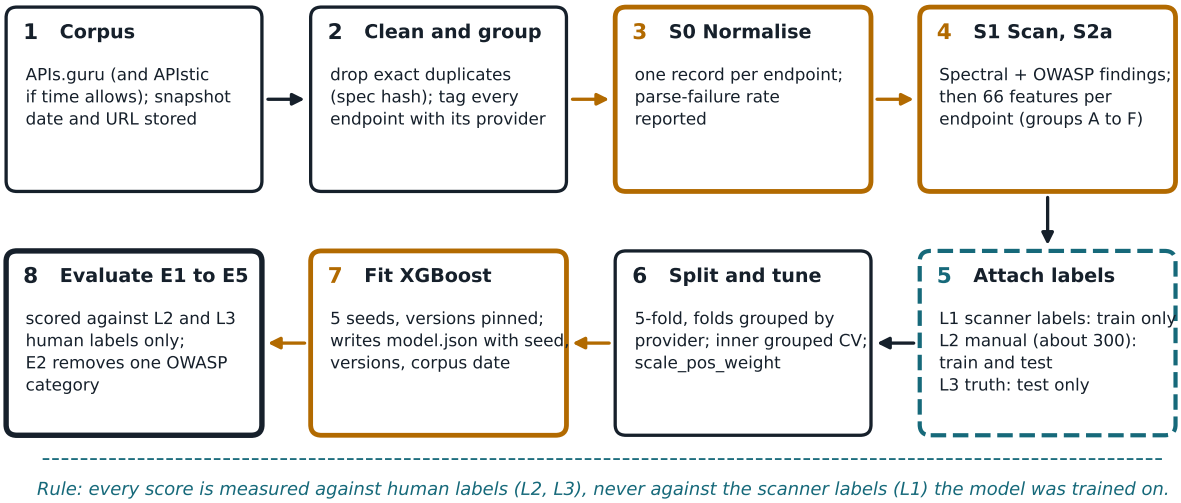


Fig. 2. Methodology in eight steps. The top row builds the dataset; the bottom row, read right to left, labels it, trains the model and evaluates it. Amber marks shared code and the model artefact; the dashed teal box marks the reference labels (L2, L3).

TABLE VII
WHAT EACH LABEL SOURCE MAY BE USED FOR

Label	Made by	Train	Evaluate
L1	Scanner	Yes	Never
L2	Three LLM passes	Train folds only	Test folds
L3	Documented flaws	No	Yes

3) **De-duplicate.** Drop files whose text hash is already present. Exact hashes miss near-copies, so the next step also protects against them.
4) **Group.** Give every endpoint a group identifier equal to its provider (the directory name in APIs.guru, the repository in APIstic). Every split in this section is made on groups, never on rows. Leakage between training and test data is a known cause of over-optimistic results in machine-learning studies [19].
5) **Specification version.** The OWASP ruleset targets OpenAPI 3.x, so Swagger 2.0 files receive fewer scanner labels for a reason unrelated to security. Scanner (L1) training labels are therefore taken from OpenAPI 3.x files only. Swagger 2.0 endpoints are still scored, and their results are reported separately. The feature `is_oas3` is constant on the training rows; it is kept so that the feature schema stays stable.

C. Labelling Protocol (T2)

Three label sources are used, and each has a fixed job (Table VII).

L1, weak labels. $y_1(e) = 1$ when the scanner reports at least one finding of severity `error` or `warn` on e, ignoring rules that fire on more than 15% of all endpoints (a pilot scan showed that, without this, over 85% of endpoints are flagged and the label carries no signal; the ignored rules are listed in the dataset statistics file). These labels are cheap and noisy. They are weak supervision in the sense of Ratner *et al.* [18], and they are used for training only.

L2, reference labels. A sample of 304 endpoints (plus a pilot of 32 that is discarded) is drawn from the de-duplicated corpus at random within strata: flagged or not flagged by the scanner, read method or write method, and small or large parent specification. Because the sample is stratified, every reported metric is weighted by the inverse of the sampling probability, so that it estimates the value on the whole corpus. *The L2 labels were produced by language models, not by people.* The procedure is:

1) Write a codebook with one decision rule per category of Section XV-A.
2) Run a pilot of 32 endpoints, revise the codebook, and discard the pilot from the 304.
3) Three independent labelling passes (two Claude passes and one Codex pass) label all 304 endpoints without seeing scanner output, model scores or the sensitive-term lexicon. The sheets given to the labellers contain only the endpoint, its specification excerpt and the codebook.
4) Report agreement before combining: pairwise Cohen's κ was 0.49 to 0.58 and Fleiss' κ was 0.54, below the target of 0.6 that was set in advance. The codebook was revised once after a pass misread an optional query-token rule; this is the only revision.
5) The final label is the majority vote of the three passes (62 of 304 endpoints, 20% positive); the category is the union of the categories of the passes that voted 1.

Agreement between labellers does not show that they are right, and agreement between language models is not independent

evidence, so L3 is the external anchor. The agreement below target, and the fact that no person checked the labels, are reported as threats to validity.

L3, ground truth. Endpoints of VAmPI and crAPI whose flaws are documented by their authors (for crAPI, the 15 challenges listed by the OWASP project [12]). The documented flaws are mapped to endpoints, and to OWASP API Top 10 2023 categories, by a language model that reads the project README and the handler source; no person reviewed this mapping. Many of these flaws live in the implementation and cannot be seen in a specification, so L3 is reported twice: for all documented flaws, and for the subset visible at design level (missing authentication, object-level access, excessive data in responses).

D. Feature Extraction (S2a)

S2a computes $d = 66$ features in six groups (Table III): A structural (18), B security scheme (14), C data sensitivity (7), D documentation (11), E scanner-derived (14) and F context (2). The implementation is a pure function with a fixed output order, no random numbers, and no null values. Two choices need a justification.

Group C uses a lexicon. It maps tokens of property names to weights 1 to 3: 3 for credentials, payment and government identifiers, 2 for tokens, sessions and sensitive personal attributes, 1 for ordinary personal data. Names are split on case changes and punctuation, so `dateOfBirth` and `date_of_birth` match the same term. The list is compiled from the OWASP API3:2023 examples and common personal-data categories, frozen before any labelling, and is English only. Property names in other languages are not matched; this is a stated limitation.

The provider is never a feature. It is used only to group splits, so the model cannot learn who wrote a specification.

E. Training Algorithm (T4)

Algorithm 1 gives the procedure. The unit of every split is a provider. The loss is the logistic loss of XGBoost [6]. The positive class is a small minority, so the positive-class weight is set from the training fold,

$$\texttt{scale_pos_weight} = \frac{N_{\text{neg}}}{N_{\text{pos}}}, \tag{3}$$

and models are compared with PR-AUC [17], F1 and the Matthews correlation coefficient [16], never accuracy.

Four baselines are trained on the same feature table and scored on the same folds: B0 the majority class; B1 the scanner itself (flagged is 1, optionally ordered by severity); B2 logistic regression on standardised features; B3 random forest. A fifth baseline, B4, asks a large language model for a risk score of the normalised endpoint in a fixed prompt at temperature 0, with the model name, version and date recorded. B4 is run only on the L2 sample, because calling it on 100,000 endpoints is not affordable, and its cost and latency are reported next to its accuracy.

Algorithm 1 Grouped training and tuning (T4)

Require: endpoints with features x, weak label y_1, reference label y_2 (where present), group g
Ensure: model f, predictions on every test fold
1: split the groups into 5 folds $P_1, \ldots, P_5$ with no group in two folds
2: **for** $i = 1$ **to** 5 **do**
3: $T \leftarrow$ endpoints with $g \notin P_i$; test set $\leftarrow$ endpoints with $g \in P_i$
4: label of each $e \in T$ is $y_2(e)$ if it exists, else $y_1(e)$
5: draw 30 hyperparameter settings at random (depth 3–8, learning rate 0.03–0.3, subsample, column sample, min. child weight, λ)
6: **for** each setting **do**
7: inner 3-fold cross-validation on T, again grouped by provider; train with early stopping on the inner validation part
8: record mean PR-AUC
9: **end for**
10: refit the best setting on all of T with `scale_pos_weight` from T
11: score the test set; for L2 and L3 endpoints keep the score and the reference label
12: **end for**
13: repeat with 5 random seeds; report mean and interval over the seeds and folds
14: fit the final model on all groups and write `model.json` with seed, tool versions, schema versions and corpus date

F. Threshold and Gate Algorithm (S3)

The threshold τ is chosen on validation folds, never on test folds. Three operating points are reported: the τ that maximises F1 (the default), the smallest τ with precision at least 0.9, and the largest τ with recall at least 0.9. No post-hoc calibration is applied, so that the score stays the logistic transform of the margin that SHAP explains; a calibration curve is reported as a diagnostic.

As specified, the gate of (2) rescans every endpoint, so a legacy API with old problems would fail every build. The gate therefore counts only endpoints that are new or changed relative to the base branch. Let S_b and S_h be the normalised base and head specifications, let $\text{key}(e) = (m, p)$ be the method and path of e, and let $h(e)$ be the hash of the normalised record without file-identity fields. Then

$$\Delta(S_h, S_b) = \big\{\, e \in E(S_h) \;:\; \nexists\, e' \in E(S_b), \\ \text{key}(e') = \text{key}(e) \wedge h(e') = h(e) \,\big\}, \tag{4}$$

$$G_\Delta(S_h, S_b) = \begin{cases} 1, & \big|\{\, e \in \Delta(S_h, S_b) \;:\; r(e) \geq \tau \,\}\big| > k, \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

The report still lists every endpoint in risk order and marks which ones were in scope. If no base exists, every endpoint is new. A renamed path counts as a new endpoint. The endpoint identifier of S0 cannot be used for matching, because it contains the specification hash and changes whenever any line of the file changes. Matching costs $O(n)$ in the number of endpoints.

G. Experiment Protocols

All results use the same grouped folds, five seeds, and a provider-level bootstrap with 1,000 resamples for 95% intervals. Two methods are compared on the same resamples.

E1, accuracy against reference labels. Algorithm 1 is run and every model is scored on the L2 and L3 endpoints of its

Algorithm 2 Runtime scan with changed-endpoint gate (S0–S3)

Require: head specification S_h, optional base specification S_b, model f, threshold τ, tolerance k
Ensure: ranked report and exit code
1: $E_h \leftarrow \text{S0}(S_h)$; $E_b \leftarrow \text{S0}(S_b)$ if S_b is given, else $\emptyset$
2: $A \leftarrow \text{S1}(S_h)$ {findings attributed to endpoints}
3: **for all** $e \in E_h$ **do**
4: $x(e) \leftarrow \text{S2a}(e, A(e), |E_h|)$; $r(e) \leftarrow f(x(e))$
5: **end for**
6: $\Delta \leftarrow$ endpoints of E_h that are new or changed relative to E_b (all if S_b is absent)
7: $n \leftarrow |\{e \in \Delta : r(e) \geq \tau\}|$
8: compute SHAP for the 10 highest-scoring endpoints; write the report ranked by $r(e)$, marking Δ
9: **return** 1 if $n > k$, else 0

test folds. The scanner is scored against the same reference labels, so the comparison with the model is fair. Per-version results (OpenAPI 3.x and Swagger 2.0) are reported separately.

E2, rule hold-out. This answers RQ2. Let $\mathcal{C}$ be the categories with at least 20 positive endpoints in L2 and L3. For each $c \in \mathcal{C}$:

1) Remove from the training set every endpoint whose only findings are in category c.
2) Recompute every group-E feature from the findings with all category-c findings deleted, so that no feature carries any trace of c.
3) Train as in Algorithm 1.
4) On test folds, take the L2 and L3 endpoints whose reference label is category c as positives, and the endpoints labelled 0 as negatives. Report PR-AUC and recall at τ.

The reference for chance is the prevalence of c, and the reference for the best case is the model trained with c included. A category counts as *generalised* when the lower end of the 95% interval of (PR-AUC minus prevalence) is above zero. The answer to RQ2 is the number of categories that generalise out of $|\mathcal{C}|$, reported with its intervals. This criterion is fixed here, before any result exists.

E3, analyst effort and speed. On the L2 sample, report precision at $k = 5, 10, 20$ and the number of findings read before all true positives are found, for the model ranking against the scanner severity ranking, with τ swept over 0.1 to 0.9. Wall-clock time of S0 to S3 is measured per component, 10 runs per specification, on specifications of 10 to 500 endpoints, which tests the requirement that the runtime lane finishes in seconds.

E4, external validity. VAmPI and crAPI are scored by the final model, which never saw them. Report the area under the ROC curve and the rank of each documented-flaw endpoint among all endpoints. Schemathesis is run against the live VAmPI container (E9) to give an independent dynamic finding set, which is compared with the ranking. OWASP ZAP was not run. The tool is used here only, never on the corpus.

E5, ablation and explanation. Remove each feature group in turn, and also remove group E alone, and report the loss in PR-AUC. Report mean absolute SHAP values globally. SHAP values for tree ensembles are computed exactly with TreeSHAP [14]. They explain the raw margin (log-odds), so the attributions sum to the margin minus the base margin, not to $r(e)$ itself. A *text-inflation test* then checks whether the model can be gamed: for L2 negatives, set the group-D length features to the 90th percentile of the training data, change nothing else, and report the mean score change and the share of endpoints that cross τ.

H. Experimental Setup

Data. The APIs.guru directory holds 2,529 APIs with 108,837 endpoints before de-duplication [8]; counts after de-duplication and parsing are reported from the frozen snapshot. VAmPI [10] and crAPI [11] are used for L3 and E4 only.

Software. Python, XGBoost, Spectral and the ruleset are pinned in a requirements file. The feature schema (`s2a-1.0`) and the lexicon (`lex-1.0`) carry version labels that are written into `model.json`, so a result can be traced to the exact code that produced it. Every random seed is fixed and recorded.

Hardware. Training and all experiments run on one CPU workstation; the tabular feature table does not need a GPU. Runtime in E3 is measured on a GitHub-hosted Ubuntu runner, which is the reference deployment, and on the team's laptop, 10 runs per specification, with the median and the range reported. Network time is excluded.

Metrics. PR-AUC, F1 and MCC for classification; precision at k and findings read for analyst effort; milliseconds per specification for speed; mean absolute SHAP value for explanation. All intervals are 95% provider-level bootstrap intervals.

Reproducibility. The code, tests, feature schema and the figure scripts are kept in one repository, and the model file used for each reported table is committed with it: `model.json` for E1 to E4, and `model_l2plus.json` (trained on the 608 reference labels of E7) for the packaged tool.

I. Threats to Validity

Label noise. L1 contains scanner errors; this is why it is never used for evaluation. **Small, model-labelled sample.** 304 endpoints give wide intervals, and the labels come from three language-model passes with Fleiss' $\kappa = 0.54$ and no human check; intervals are reported rather than hidden, and L3, whose flaws are documented by the VAmPI authors, is the external anchor (the flaw-to-endpoint mapping was done by a language model reading the handler source). **Specification versus implementation.** Design-level scores cannot see implementation flaws; E4 measures the gap. **Corpus bias.** APIs.guru is dominated by public commercial APIs and may not represent internal APIs. **Language.** The lexicon is English only. **Tool drift.** A new Spectral or ruleset version changes L1 and the baseline together, so versions are pinned. **Language-model baseline.** Output can change between model versions; the exact model and date are recorded, and B4 is a point of comparison, not part of the system.

TABLE VIII
E1: ACCURACY ON L2 (MEAN OF FIVE SEEDS)

Method	PR-AUC	ROC-AUC
Always positive	0.21	0.50
Scanner only (B1)	0.47	0.69
Logistic regression	0.33	0.72
XGBoost (APIGUARD)	0.42	0.70

TABLE IX
E2: HELD-OUT CATEGORY, PR-AUC AGAINST PREVALENCE

Cat.	Pos.	Prev.	PR-AUC	ROC-AUC	Generalised
API2	54	0.18	0.22	0.57	No
API3	24	0.10	0.15	0.50	No
API5	34	0.09	0.37	0.91	Yes

J. Cut Order if the Schedule Slips

Items are cut in this order: the APIstic scale-up, the 42Crunch audit score, and B4 on any sample larger than L2. E1 to E5, and in particular E2, are never cut.

XVI. RESULTS

All numbers come from the frozen run of 6 November 2026 on 80,974 endpoints from 665 providers, evaluated on the 304 L2 endpoints (62 positive, 20%). Intervals are 95% provider-level bootstrap intervals.

A. E1: Accuracy

Table VIII gives the mean over five seeds. The model is twice as good as always guessing the positive class (PR-AUC 0.21) and clearly better than logistic regression (difference 0.014 to 0.189). It does *not* beat the scanner alone: the difference to the scanner is -0.20 to $+0.16$, which includes zero.

B. E2: Rule Hold-out (RQ2)

Only 3 of the 10 categories had the 20 positives needed for a test. The model generalised on one of them (Table IX). The answer to RQ2 is therefore **1 of 3 tested categories**, and the other seven could not be tested with this sample.

C. E3: Effort and Speed

Ranking by model score did not reduce reviewer effort compared with scanner severity: precision at 5, 10 and 20 was 0.20, 0.30 and 0.20 for the model against 0.60, 0.40 and 0.55 for the scanner. The precision and effort numbers use the out-of-fold scores of E1, so no test endpoint was seen in training. The best F1 over the threshold sweep was 0.47. Run time from S0 to S3 was 1.5 s for the 14-endpoint VAmPI specification and 10.5 s for the largest (422 endpoints). The Spectral scan (S1) takes 1.4 s and 5.6 s, normalisation (S0) 0.07 s and 2.7 s, and the feature builder (S2a) 0.004 s and 2.1 s; the model (S2b) and the SHAP explanation (S2c) together take under 25 ms in every run.

D. E4: External Check on VAmPI and crAPI

VAmPI has 14 endpoints, 10 with a documented flaw and 5 visible at design level. The ROC-AUC over all flaws was 0.48 (PR-AUC 0.77 against a prevalence of 0.71) and over the design-visible flaws 0.22; the design-visible endpoints ranked 8th to 12th of 14. crAPI has 44 endpoints, 15 mapped to the 15 challenges that the OWASP project documents [12], and 3 of them visible at design level. The ROC-AUC over all flaws was 0.39 (PR-AUC 0.30 against a prevalence of 0.34) and over the design-visible flaws 0.16; those three endpoints ranked 31st, 38th and 40th of 44.

This is a negative result on both applications: the model does not rank flawed endpoints above others, and it is worse than a random ranking on the flaws that a specification can show. The ranked tables show why. The highest scores go to authentication and write endpoints that the scanner flags many times, and the lowest scores go to read (GET) endpoints that declare no authentication, which are the clearest design-level flaw (`/users/v1/_debug` in VAmPI, `receive_report` in crAPI). A model trained to agree with scanner and language-model labels inherits whatever they under-weight. Two cautions apply: each application is small, so no interval is given, and the flaw-to-endpoint mapping was made by a language model from the project documentation and handler source (Section XV-A). The dynamic comparison with Schemathesis is reported in Section XVI-J for VAmPI only; OWASP ZAP was not run and crAPI was not tested dynamically. Section XVI-F tests a likely cause.

E. E5: Ablation and Explanation

Removing the structural group A cost the most (PR-AUC 0.44 to 0.30). The other groups changed it by at most 0.04. Mean absolute SHAP was highest for the security-scheme group B, then scanner-derived E and structural A. The text-inflation test did not game the model: setting description lengths to the 90th percentile moved the mean score by -0.011 and only 0.7% of negatives crossed τ.

F. E6: Training Only on the Reference Labels

The E4 failure has a likely cause in how E1 to E4 were trained. About 81,000 of the training labels are scanner (L1) labels, and L1 ignores every rule that fires on more than 15% of endpoints. That list includes `owasp:api2:2023-read-restricted`, the rule for a read endpoint without authentication. So the model was taught that such endpoints are normal. The reference labels say otherwise: among the 304 L2 endpoints, 47% of those with no security requirement are positive against 5% of those with one. E6 therefore trains the model *only* on the 304 reference labels, with the same provider-grouped five-fold protocol and five seeds, and compares it with the scanner on the same test folds. Only this one variant was run. It was motivated by the E4 result and by the label rates above, so E4 is no longer a clean external test for it; the E6 test folds are, because they are out-of-fold.

TABLE X
E6: TRAINED ON L2 ONLY, OUT-OF-FOLD ON L2 (MEAN OF FIVE SEEDS)

Method	PR-AUC	ROC-AUC	F1	MCC
Scanner only (B1)	0.47	0.69	0.43	0.26
Logistic regression	0.47	0.81	0.52	0.38
XGBoost, L2 only	0.48	0.80	0.52	0.39

TABLE XI
E7: XGBOOST ON THE ORIGINAL TEST SET, MEAN OF FIVE SEEDS

Training labels	PR-AUC	ROC-AUC	F1	MCC
Scanner only (B1)	0.47	0.69	0.43	0.26
L2 only (E6, 304)	0.48	0.80	0.52	0.39
L2 + extra (608)	0.52	0.81	0.51	0.40

On the seed-averaged out-of-fold scores the ROC-AUC of XGBoost is 0.84 against 0.69 for the scanner, and the provider-level bootstrap interval of the difference is $+0.01$ to $+0.33$, which excludes zero only narrowly. The PR-AUC difference is -0.20 to $+0.37$ and includes zero. On the two external applications the L2-only model gave ROC-AUC 0.86 on VAmPI against 0.48 before (design-visible flaws 0.72 against 0.22), and 0.39 on crAPI against 0.39 before (design-visible flaws 0.63 against 0.16). So it fixed VAmPI and not crAPI. The effort measures of E3 are repeated with the out-of-fold scores of this model in Section XVI-I. Because the labels are language-model labels, E6 measures agreement with those labels; the two applications are the only evidence that does not depend on them.

G. E7: Doubling the Reference Labels

To test whether label count is the limit, 304 further endpoints were labelled by one language-model pass with the same codebook and the same stratified draw, excluding every endpoint of the test sample and the pilot (67 positives, 22%). They are used for training only; the test set is still the original 304 endpoints, scored out-of-fold, and the extra endpoints of a provider that is in a test fold are not used for that fold. Because it is a single pass, no agreement score exists for these labels.

Doubling the labels did not change the result measurably: the difference to E6 is -0.16 to $+0.22$ in PR-AUC and -0.05 to $+0.05$ in ROC-AUC. Against the scanner, the seed-averaged ROC-AUC is 0.84 against 0.69, with a difference of $+0.03$ to $+0.32$, and the PR-AUC difference is -0.11 to $+0.34$. On the external applications the model trained on 608 labels reached ROC-AUC 0.79 on VAmPI and 0.37 on crAPI. So more labels of this kind did not fix crAPI, and label count alone is not the limit; label quality and coverage of the flaws that crAPI contains (mostly implementation flaws) are more likely limits.

H. The Packaged Tool

The command line was packaged as a single Windows executable with PyInstaller (about 410 MB, because it carries XGBoost and SHAP). It needs Node.js and the Spectral package next to it for the scanner step. Run on VAmPI (14 endpoints) and crAPI (44 endpoints) with `model_l2plus.json` and the model's own threshold $\tau = 0.53$, it printed the ranked endpoints with their top SHAP reasons and the scanner rules that fired, and ended with "GATE: FAIL" and exit code 1 on both, the expected verdict for applications built to be vulnerable. The gate was not tested on a clean specification, so its false-alarm rate on that kind of input is known only from the E1 and E3 measurements above.

TABLE XII
E8: RANKING ON THE TEST SET (MODEL TRAINED ON THE 608 REFERENCE LABELS OF E7)

Ranking	PR-AUC	ROC	P@5	P@10	P@20	Read
Scanner only	0.47	0.69	0.60	0.40	0.55	272
Model only	0.52	0.81	0.64	0.60	0.57	279
Scanner + model	0.55	0.80	0.88	0.76	0.66	245

I. E8: Scanner and Model Combined

E3 asked whether the model ranks findings better than the scanner and the answer was no. The design places the model *after* the scanner, so E8 tests the combination: the score is the mean of the percentile rank given by the scanner (severity, then rule count) and the percentile rank given by the model. This one setting was fixed in advance and no weight was tried. All scores are out-of-fold on the original 304 test endpoints, averaged over the five seeds; precision at k and the findings read before the last true positive follow the definitions of E3.

Against the scanner the combination gains $+0.03$ to $+0.30$ in PR-AUC and $+0.05$ to $+0.24$ in ROC-AUC (provider-level 95% intervals, both excluding zero), and a reviewer who reads the top 10 finds 7.6 true positives on average in expectation against 4.0 (precision 0.76 against 0.40). Against the model alone the differences are not significant (-0.13 to $+0.19$ in PR-AUC). The model trained only on 304 labels (E6) gives a smaller gain whose PR-AUC interval includes zero (-0.16 to $+0.27$). Two caution points: this is the best of the combinations reported here only in the sense that it is the one we specified, but the two training policies and the two metrics make four comparisons, so the intervals should be read as indicative; and precision at k has no interval because it is computed on a single sample of 304 endpoints with 62 positives.

J. E9: Dynamic Check with Schemathesis on VAmPI

To get one finding set that does not come from our labels, Schemathesis 4.33.2 (official Docker image) was run against a live VAmPI container (freshly populated database, a valid login token, all checks, up to 50 examples per operation, 805 test cases, 14 of 14 operations). It reported 18 unique failures, on 8 of the 14 scored endpoints and on 3 OPTIONS operations that the model does not score. Schemathesis checks server errors, schema conformance, undocumented responses and state handling; it has no check for logic flaws such as broken object-level authorisation. Two readings of its output were prepared before it was

compared with the model: *any* failure on an operation (8 of the 14 endpoints), and *strict* failures only (server error, response violating the schema, or created resource not found: 2 endpoints, `PUT /users/v1/{username}/password` and `GET /books/v1/{book_title}`).

The agreement is weak. Against the 10 documented flaws, Schemathesis flags 5 endpoints under the *any* reading (recall 0.50) and 2 under the *strict* reading (recall 0.20), while the four endpoints above the gate threshold of the model trained on 608 labels are all documented flaws (`GET /users/v1/{username}`, `POST /users/v1/login`, `POST /users/v1/register`, `GET /users/v1/_debug`). Taking the dynamic finding as the label, the model score gives a ROC-AUC of 0.25 (*any*) and 0.27 (*strict*) over 14 endpoints, that is no agreement, and slightly below chance. The two tools find different things: the model ranks endpoints whose design resembles known OWASP risks, whereas Schemathesis finds endpoints that misbehave under malformed input. The endpoint the model ranks first (`GET /users/v1/{username}`) is not flagged by Schemathesis at all. With 14 endpoints and 2 to 8 positives this is an observation, not a test. It means the dynamic tool cannot confirm the ranking, and it also cannot refute it, because it does not look for the flaws that the ground truth lists.

K. Discussion and Limitations

The honest conclusion has two parts. Trained on scanner labels (E1 to E4), APIGUARD matches the scanner at similar accuracy and the same cost, generalises to one unseen category, and does not beat the scanner. Trained only on the reference labels (E6), it beats the scanner on ROC-AUC (difference $+0.01$ to $+0.33$) and on F1 and MCC, but not significantly on PR-AUC, and it still ranks crAPI flaws no better than chance, even with twice as many reference labels (E7). Combined with the scanner (E8), it ranks better than the scanner alone in both PR-AUC and ROC-AUC, which is the use the design intends. Four things limit this. (i) The reference labels were produced by language models, with Fleiss' $\kappa = 0.54$ and no human check, so noise in them caps every score. (ii) Only 62 positives exist, so most categories were untestable. (iii) L1 labels are the scanner's own, so a model trained on them can at best approximate it. (iv) On the two external applications the scanner-trained model ranked flawed endpoints no better than chance (E4), and the L2-trained model did well on VAmPI but not on crAPI (E6), so the claim is limited to the corpus labels, and the dynamic ZAP/Schemathesis comparison was not run. A larger, human-labelled sample and more vulnerable-by-design applications would be the first things to add.

Sensitivity to the labels. The analysis was repeated with an earlier key made from two passes plus an adjudication (46 positives). E1 was similar (XGBoost 0.46, scanner 0.43, difference -0.21 to $+0.30$). E2 changed: with that key only API2 and API5 had enough positives, and both generalised (2 of 2). So the E2 answer depends on the label key. API5 generalises under both keys; API2 does not under the majority key. We report the majority-vote result as the main one, because it was fixed before the sensitivity run, and treat RQ2 as unresolved at this sample size.

XVII. Conclusion

This deliverable has specified the APIGUARD system model: three lanes separated by when they run and how far they are trusted, a runtime path of four components from specification to CI exit code, an offline path that produces the model artefact those components load, and a feedback path that returns human verdicts to the label store without ever closing a loop into the model. The notation and the gate rule in (1) and (2) fix what the implementation must compute, and the interface contracts fix what each component must guarantee. Section XV turns this model into the proposed methodology and algorithm: the label protocol that keeps scanner labels out of every evaluation, the feature extraction, the grouped training procedure, a gate that judges only new or changed endpoints, and the protocol for the rule hold-out experiment that the whole design rests on. The experiments of Section XVI show that the model matches but does not beat the scanner, and that its rule hold-out generalisation is limited to one of three testable categories.

References

[1] S. Serbout and C. Pautasso, "APIstic: A large collection of OpenAPI metrics," in *Proc. 21st Int. Conf. Mining Software Repositories (MSR)*, Lisbon, Portugal, 2024, pp. 265–277, doi: 10.1145/3643991.3644932.

[2] C. Cheh and B. Chen, "Analyzing OpenAPI specifications for security design issues," in *Proc. 2021 IEEE Secure Development Conf. (SecDev)*, 2021, pp. 15–22, doi: 10.1109/SecDev51306.2021.00019.

[3] F. Tanveer, F. Iradat, W. Iqbal, and A. Ahmad, "Towards secure APIs: A survey on RESTful API vulnerability detection," *Computers, Materials & Continua*, vol. 84, no. 3, pp. 4223–4257, 2025, doi: 10.32604/cmc.2025.067536.

[4] A. Decrop, M. Vandeloise, P. Heymans, and G. Perrouin, "OASQuali: Automated quality analysis of OpenAPI specifications," in *Proc. Int. Conf. Web Engineering (ICWE)*, 2026, pp. 94–108, doi: 10.1007/978-3-032-29372-5_7.

[5] L. Flynn, W. Snavely, D. Svoboda, N. VanHoudnos, R. Qin, J. Burns, D. Zubrow, R. Stoddard, and G. Marce-Santurio, "Prioritizing alerts from multiple static analysis tools, using classification models," in *Proc. 1st Int. Workshop Software Qualities and Their Dependencies (SQUADE)*, Gothenburg, Sweden, 2018, pp. 13–20, doi: 10.1145/3194095.3194100.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, Long Beach, CA, USA, 2017, pp. 4765–4774.

[8] APIs.guru, "OpenAPI directory." [Online]. Available: https://apis.guru/

[9] Stoplight, "spectral-owasp-ruleset." [Online]. Available: https://github.com/stoplightio/spectral-owasp-ruleset

[10] erev0s, "VAmPI — vulnerable REST API." [Online]. Available: https://github.com/erev0s/VAmPI

[11] OWASP, "crAPI — completely ridiculous API." [Online]. Available: https://github.com/OWASP/crAPI

[12] OWASP, "crAPI challenge solutions." [Online]. Available: https://github.com/OWASP/crAPI/blob/main/docs/challengeSolutions.md

[13] OWASP, "API security top 10 — 2023 edition." [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x11-t10/

[14] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020, doi: 10.1038/s42256-019-0138-9.

[15] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960, doi: 10.1177/001316446002000104.

[16] D. Chicco and G. Jurman, “The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation,” *BMC Genomics*, vol. 21, art. 6, 2020, doi: 10.1186/s12864-019-6413-7.

[17] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, art. e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[18] A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré, “Snorkel: Rapid training data creation with weak supervision,” *Proc. VLDB Endowment*, vol. 11, no. 3, pp. 269–282, 2017, doi: 10.14778/3157794.3157797.

[19] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, art. 100804, 2023, doi: 10.1016/j.patter.2023.100804.